



FIELD GUIDE

The future of identity

Orchestrating secure access
for the agentic era

Table of Contents

Introduction

The identity gap with AI agents and why traditional IAM is out of step with autonomous systems.

Chapter 1:

Rethinking Identity for Autonomous Agents

Why AI agents break traditional identity assumptions — and how identity orchestration solves for the specific challenges of disconnected, hybrid environments.

Chapter 2:

From Agent Sprawl to Structured Governance

The rise of unmanaged AI agents and how an agent fabric and registry establish visibility and control.

Chapter 3:

Identity That Works Anywhere Agents Run

Why legacy access control can't keep up with dynamic AI agent behavior — and the shift to context-aware, delegated access policies.

Chapter 4:

Escaping the Human-Centric Identity Trap

How federated identity orchestration brings continuity, control, and auditability across disconnected and hybrid environments.

Chapter 5:

Governing Identity at Machine Scale

Solving the provenance problem with On-Behalf-Of (OBO) delegation and execution graphs that trace AI agent activity end-to-end.

Chapter 6:

Extending OAuth for the Agentic Future

Addressing machine identity sprawl with Just-in-Time (JIT) provisioning and lifecycle-aware identity management for agents.

Chapter 7:

Giving Agents the Identity They Deserve

How a unified agent identity fabric enables consistent agentic behavior governance across clouds, platforms, and orchestration tools.

Chapter 8:

Designing Identity for the Agentic User Flow

Moving beyond human-centric IAM with AI agent-native architecture built for speed, scale, and zero-trust enforcement.

Conclusion:

A New Identity Model for an Autonomous Future

Why the future of IAM belongs to those who can govern at the speed of agentic automation.

Join the Preview

Learn how to get early access to AI agent-native IAM with Strata's Mavericks platform.

Introduction

The future of identity: Orchestrating secure access for the agentic era

Enterprise identity systems were built for a different era: one where flesh-and-blood human users logged in, stayed logged in, and followed predictable access patterns. That world has changed rapidly with the introduction of agentic AI, and we are not ready.

AI agents now operate at a different scale and speed, executing tasks for human users across public clouds, edge networks, on-premises providers, and disconnected systems. Yet they're running with little visibility and without any identity governance at all, creating huge security risks.

What is an agentic identity?

An **agentic identity** is a digitally verifiable identity assigned to an AI agent. Unlike traditional non-human identities (NHIs), which are static and long-lived (like service accounts), agentic identities are:

- **Ephemeral** – spun up and destroyed in seconds
- **Delegated** – acting on behalf of a user, app, or another agent
- **Context-bound** – linked to a task, intent, or environmental trigger

These AI agents are already embedded in your workflows, as they pull data, trigger actions, and make decisions in places your current stack can't fully govern. They don't have user accounts. They don't wait for access approvals. And they leave behind no clear record of what they did or who enabled them. It's a transition that's already well underway inside most enterprises.

The chapters ahead break down the architecture gaps exposed by AI agent-driven automation and present a practical path forward. Through eight focused problem-solution chapters, we'll lay out how organizations can rethink identity from the ground up, introducing orchestration, policy, and lifecycle tools purpose-built for autonomous actors.

Chapter 1:

Rethinking identity for autonomous AI agents

In the age of agentic AI, systems that don't just assist they act. Agents can make decisions, carry out tasks, and adapt to changing contexts — autonomously. But with autonomy comes accountability. And the question becomes: who is acting?

To answer that, we need a new identity model built not for humans, but for artificial agents.

Problem: The hidden identity crisis in hybrid AI agent deployments

AI agents don't live in tidy, centralized environments. They're scattered across public clouds, private infrastructure, disconnected networks, ships, and factory floors.

Yet traditional IAM assumes policy-based controls and human-user-driven access, an architecture that breaks the moment an agent spins up in a place without connection to the central IDP. Worse, many IAM tools treat AI agent identity as an afterthought. There's no runtime delegation, provenance tracking, task traceability, or enforcement at the edge.

Deployment challenges with AI agents

Some of the challenges enterprises find with deploying AI agents across hybrid environments include:

Legacy IAM doesn't cover hybrid environments

Most identity tools were built for cloud-first, human users, not for agents spread across disconnected, on-prem, and multi-cloud infrastructures. This leads to insecure workarounds like hardcoded secrets and unauthenticated agents.

Cloud-native tools break in regulated or air-gapped settings

In industries bound by strict compliance or operational constraints, agents must run locally. But modern IAM systems often can't follow, leading to token issuance, access control, and auditability failures.

No identity model for agent workflows

Agents now participate in core business operations, yet lack proper identity linkage, scoped access, or policy enforcement, leaving enterprises unable to trace or control their actions.

Little to no visibility into agent behavior

Security teams often lack the ability to track agents' actions, who authorized them, and whether they followed policy. Without observability, incident response and compliance are compromised.

Fragmented policy enforcement

Hybrid deployments require a coordinated identity policy. Instead, teams are stuck managing static credentials, duplicating configs, and manually stitching logs, creating governance silos.

Agents are scaling faster than humans

Forecasts suggest organizations will manage up to 80x more agents than people. Traditional IAM architectures can't keep up with this velocity, volume, or variability.

Enterprises need a change in strategy for identity to incorporate agents to avoid being buried under a volume of unmanaged machine identities they can't secure or control.

Human identity vs. machine identity vs. agentic identity

Agentic identity is a fundamental shift from how identity has been handled for humans or even machine identities (what we call “non-human identities” or NHIs). NHIs are often long-lived and managed like infrastructure: think of a backend service with a static key in a key vault. Agentic identities, in contrast, are active actors in a runtime workflow.

Property	Human Identity	Traditional Machine Identity (NHI)	Agentic Identity
Lifespan	Long-lived (years)	Long-lived (days/months)	Ephemeral (can range from seconds to minutes to longer)
Origin	Manual creation, enrollment	Provisioned via scripts or platforms	Agent identity in IDP JIT-generated based on policy
Authentication	MFA, SSO, passkeys	API key, mTLS cert, SPIFFE SVID	PKCE, SVID, OAuth bearer or DPoP
Access Control	Role-based (RBAC/ABAC)	Scoped service roles	Task-bound, dynamic scopes
Logging & Auditing	Tied to user session	Often limited, coarse-grained	Rich telemetry, chain of delegation
Governance	IGA, certifications, approvals	Manual or SCIM-based	Policy-driven, dynamic lifecycle

Solution: Identity Orchestration for AI agents

Today’s IAM systems are designed around human-centric workflows: long-lived credentials, role-based access, and predictable sessions. They don’t account for AI agents’ “bursty,” delegated, and dynamic nature.

Why enterprises need agentic identity now

AI agents like copilots, bots, and autonomous systems are making real decisions — issuing refunds, rebalancing infrastructure, and executing trades. Like human users, they authenticate, get authorized, and have their actions logged.

The risks created when agentic identity lacks visibility and control are:

- **Credential sprawl** increases as agents reuse human tokens.
- **Accountability disappears**—there’s no way to trace who did what.
- **Auditing fails**, exposing organizations to compliance risk.

Humans and agents must operate together, not in isolation. An agent might initiate a task, but a human needs to approve or oversee it. Agentic identity ensures that these interactions are secure, traceable, and governed.

The “6 A’s of identity” for agentic AI

To ensure trust and accountability, identity systems must support delegated authority (e.g., OAuth OBO) and clearly log who did what, on whose behalf. Identity must evolve across the six domains of identity and access management to manage AI agents effectively:

- **Authentication** – via PKCE, SPIFFE/SVID, or JWT.
- **Access control** – scoped to specific tasks, APIs, and contexts.
- **Authorization** – including delegated flows using OAuth On-Behalf-Of.
- **Auditing** – full traceability of every agent action and decision chain.
- **Administration** – with just-in-time (JIT) provisioning and lifecycle control.
- **Availability** – resilient identity enforcement across clouds, on-prem, and air-gapped environments.

As enterprises scale AI deployments, agentic identity will become the foundation for safe and trusted autonomy, treating agents not as scripts or extensions, but as accountable digital actors.

What’s needed next: A new foundation for agent-first identity

Solving these challenges requires more than patching IAM gaps — it demands a new model that treats AI agents as first-class identities. A hybrid Identity Orchestration platform is needed to unify access and policy across clouds, legacy systems, and air-gapped environments for both human and machine actors. With orchestration, enterprises can enforce policy and monitor agent behavior at scale; without it, the risk only grows.

Chapter 2:

From AI agent sprawl to structured governance

AI agents are becoming central to how work gets done — from handling customer service chats to triggering infrastructure automation. But identity infrastructures can't handle agents.

Problem: Identity architectures weren't built for this

AI agents are not only in carefully planned deployments but also in ungoverned corners of the enterprise. Human users get profiles in identity providers. Machines get service accounts. But where do agents go? We need to figure this all out — fast!

Here are the specific, systemic problems that traditional identity systems weren't built to solve — and why they're now slowing down secure enterprise adoption of agent-based AI.

No system of record for AI agents

Since agents aren't provisioned in an IDP, their credentials are hardcoded or shared, and often aren't reviewed by security teams at all. Here's where they appear:

- Low-code automations running in CI/CD pipelines.
- Scripts written by developers with embedded tokens.
- External LLMs wired into production systems.
- Bots spun up in cloud services without a trace.

Without an authoritative registry for agent identities, agents become untracked actors in your environment — able to perform actions with no attribution, scope boundaries, or accountability.

AI agents can't be discovered or organized at runtime

Security teams can't discover agents at runtime, and even if they could, there's no central authority to categorize or contain them. As a result, organizations are faced with:

- Over-scoped permissions — agents granted admin-level API keys by default.
- Hardcoded credentials — often never rotated, rarely revoked.
- Shadow agents — automations that bypass identity review and oversight entirely.

Without structured governance, the enterprise identity surface is mushrooming with untracked actors.

AI agent identity is fragmented, over-permissioned, and ungoverned

Most agents operate with broad, static credentials — admin API keys or shared user tokens — making them high-risk targets. Permissions are hard to audit or revoke without scoped tokens or a centralized policy.

Agent identity is fragmented across clouds (Azure, AWS, GCP) and on-prem platforms, each with different (or no) identity models. There's no shared standard for authentication, authorization, or access control. This creates blind spots:

- No visibility into who delegated what to which agent
- No way to trace intent or validate actions
- No consistent policies across environments

Shadow agents (unreviewed scripts, low-code bots, or external LLMs) often run outside governance entirely, with no enrollment, tracking, or access control.

What's missing is a unified abstraction layer for identity that connects humans, agents, and apps through delegation, scoped credentials, and runtime enforcement. Without it, IAM breaks under the weight of agentic scale.

Solution: The agent fabric and registry

To restore control, organizations need a dedicated identity layer for AI agents: the agent fabric. It treats agents not as human proxies or static services but as first-class entities with their own lifecycle, purpose, and policy scope.

What is an agent fabric, and why is it necessary?

An agent fabric is an identity security control plane purpose-built for AI agents. Think of it as a programmatic control plane purpose-built to manage AI agents' identities, behaviors, and boundaries at scale.

At the center of the agent fabric is the agent registry — a dynamic catalog that discovers and governs agents across clouds, frameworks, and platforms of origin. The registry provides the missing structure by:

- Discovering agents as they're instantiated in CI/CD pipelines, API workflows, or orchestration platforms.
- Classifying agents by business unit, function, and risk level.
- Tracking delegation chains, scopes, TTLs, and execution history.
- Integrating with IDPs to ensure every agent has a verifiable identity record.

An agent fabric answers a critical gap in today's AI infrastructure: AI agents don't live in a single system. They span LLM frameworks, API runtimes, CI/CD environments, and cloud-native services, with no centralized way to manage identity. The agent fabric solves this by acting as a unifying layer across platforms. With this infrastructure in place, agents can be governed like users.

Agent fabric vs. identity fabric vs. app fabric

Identity Orchestration unifies identity across humans, apps, and now AI agents, making runtime policy, audit, and Zero Trust enforcement dynamic and scalable. To understand where the agent fabric fits, consider the full stack of identity abstraction layers:

<i>Fabric type</i>	<i>Description</i>
Identity fabric	An abstraction over multiple identity providers (IDPs), assurance levels, authentication mechanisms, and risk signals. Makes policy and login flows portable.
App fabric	A layer that spans apps and APIs to provide consistent identity governance, Zero Trust controls, and access visibility, regardless of app architecture.
Agent fabric	A new abstraction layer that manages agents across clouds and runtimes, enabling identity, policy, and observability for ephemeral, autonomous systems.

Just like the identity fabric federates human authentication across providers, the agent fabric federates agent identity across platforms — and enforces consistent policies, scopes, and audit rules. It becomes the interoperability layer between distributed agents and your internal security policies.

The agent fabric is not a luxury — it's a requirement for operating agentic systems safely, at scale, and in compliance with security frameworks like Zero Trust, NIST CSF 2.0, and GDPR.

Chapter 3:

Identity that works anywhere AI agents run

AI agents don't run in a neat, uniform environment. They exist across public clouds, private data centers, disconnected networks — even on ships and factory floors. Yet how we authenticate, authorize, and govern them still assumes a centralized, cloud-connected world. We need to make hybrid work for AI agents.

Problem: Access control can't keep up with AI agent behavior

As AI continues to proliferate into the enterprise, increasingly autonomous agents continue to evolve, adapt, and operate across domains. Legacy access models are breaking down fast.

RBAC (Role-Based Access Control) and even static ABAC (Attribute-Based Access Control) assume predefined roles, predictable behaviors, and long-lived entities. But agentic AI turns that model on its head.

AI agents don't:

- Hold static roles.
- Operate within one environment.
- Stay alive for hours, let alone days.

Instead, they:

- Spin up dynamically in response to triggers.
- Change context mid-execution based on input or delegation.
- Move laterally across environments and workloads.
- Are ephemeral and often only exist for a few seconds or minutes.

The consequences here can be dire. AI Agents are routinely over-permissioned and given broad standing credentials that last far longer than needed. Plus, delegation chains are unclear or absent, so there's no way to trace back who initiated what, or why, especially in multi-agent workflows. Often, runtime context isn't considered, so the AI agent executing a task may no longer do so under appropriate conditions.

Policies built for predictable human behavior fail in the face of machine-speed autonomy.

What does hybrid identity mean today?

The term "hybrid" has evolved beyond "on-prem + cloud." In the agentic era, a hybrid architecture means:

- Public cloud platforms (e.g., Azure, AWS, Google Cloud)
- Private clouds and on-premises infrastructure
- Air-gapped or disconnected environments (DDIL, tactical edge)
- Multiple identity providers (IDPs) in use across different domains
- Cross-agent platform compatibility — AI agents running on frameworks like ChatGPT, LangChain, Azure Agent Foundry, N8N, and CrewAI

Why some things will always stay on-premises

Even as cloud adoption accelerates, some mission-critical workloads and datasets cannot — and will not — leave the premises because of:

- Regulatory constraints (e.g., financial services, defense, healthcare)
- Data residency and sovereignty (especially in GDPR- and HIPAA-covered regions)
- Latency-sensitive systems (manufacturing lines, trading engines, logistics systems)
- Operational control and uptime SLAs for critical systems

The identity systems meant to govern agents are stuck in a centralized past. That's not just a technical gap—a security, compliance, and operational risk that will only grow as agents scale faster than humans.

To solve this, we need more than incremental change. We need a new architectural mindset that treats hybrid environments not as edge cases but as the default.

Solution: Context-aware, dynamic access policies

In this new landscape, hybrid deployment isn't a deployment option — it's an operational imperative for security, resilience, and compliance. The identity of AI agents must be distributed and dynamic, as well as the agents themselves.

A simple solution may be to expand access models manually. But because that's inefficient, the key is redesigning how access is granted, evaluated, and revoked at runtime. Modern organizations need AI agent-native access control. Which requires:

- Just-in-Time (JIT) provisioning of AI agent identities to existing IDPs.
- Runtime context evaluation is tied to task, origin, and environment.
- Delegation-aware authorization that clearly defines who the AI agent is acting for and within what limits.

In hybrid environments, agents must run locally, often within secured infrastructure where the enterprise has full control over identity systems, policy enforcement, and data access. This is where air-gapped architectures become essential.

The role of air-gapped architectures in agent security

An air-gapped deployment can be described as a disconnected, often classified, environment where no inbound or outbound API communication is allowed. These environments are critical for:

- Defense and national security systems
- Critical infrastructure (e.g., financial infrastructure, utilities, emergency response)
- Remote deployments (e.g., ships, satellites, border outposts)

AI agents running in these zones must operate independently, with no dependence on cloud-hosted IDPs, policy engines, or data providers. This introduces a new challenge: how do you give AI agents identity, access, and authorization in a disconnected runtime?

That's why Strata built Mavericks Identity Layer for Agentic / Artificial Identities — to deliver the identity layer that works anywhere your agents run.

Hybrid Identity Orchestration with Mavericks

With Strata's Mavericks platform, identity and access policies are packaged and deployed locally, and OAuth tokens are minted on-prem and bound to specific agents and scopes. Here's what that looks like:

1. An AI agent is spun up in response to a user's task-oriented command.
2. A temporary identity is created on the fly, tied to the specific human user and narrowly scoped task.
3. A short-lived, scoped token is issued and is valid only for the current operation.
4. The AI agent completes the task.
5. The identity and permissions are revoked immediately after.

The system evaluates the following questions: What agent is this? Who delegated the task? What policy governs the requested action? And has anything changed in the risk posture since issuance?

Maverics can integrate with CAEP (Continuous Access Evaluation Protocol), enabling access decisions to adapt mid-task and act as a Policy Enforcement Point (PEP) for API security. If an agent strays out of policy, access is revoked immediately without waiting for the token to expire.

This model supports agents acting:

- On-behalf-of (OBO) users in sensitive workflows (e.g., HR bots, finance agents).
- In cross-domain environments (e.g., AWS Lambda calling GCP services).
- In runtime frameworks like LangChain, N8N, or CrewAI.

Access control is no longer static or manually assigned. It's bound to intent, task, and context and can be enforced dynamically at machine speed.

Chapter 4:

Escaping the human-centric identity trap

Most enterprise IAM architectures weren't built to address how identity works today. What used to be a neatly contained identity perimeter is now scattered across clouds, platforms, and disconnected operational zones, each with its own standards, systems, and silos.

Problem: IAM breaks across deployment boundaries

While agents now act with independence and intent, their identity infrastructure is stuck in the past. Most security and IAM tools assume static users, predictable sessions, and cloud-connected environments. None of that applies when you're dealing with autonomous agents that: Operate independently, make real-time decisions, act on behalf of others, scale to thousands of instances per application.

The identity crisis at the heart of the AI agent revolution

This misalignment is creating a fast-growing identity crisis in AI adoption. Identity is mobile, and it's splintered. And AI agents now operate everywhere:

- Azure, AWS, and GCP, each with different identity management tools and APIs.
- On-prem environments running legacy IAM that weren't designed for automation or scale.
- CI/CD pipelines where ephemeral workloads may not have consistent or durable identity.
- Disconnected environments, including DDIL (disrupted, disconnected, intermittent, and limited) air-gapped systems, where cloud IAM doesn't function at all.

Why traditional IAM fails agentic AI

AI agents fundamentally change how work gets done, but identity systems haven't caught up. Built for long-lived human users, legacy IAM struggles with the speed, scale, and delegation patterns of agentic AI. Let's explore some of the reasons traditional IAM doesn't work with agentic AI.

Human IAM doesn't fit AI agents

Legacy IAM relies on long-lived accounts, passwords, and manual provisioning. AI agents need ephemeral, just-in-time credentials and scoped, runtime access, leading many teams to insecure workarounds.

OAuth and API keys weren't built for agents

OAuth assumes login and consent—agents don't do either. They act on behalf of others, spin up rapidly, and chain requests, making traditional tokens inadequate for secure delegation and traceability.

Static access control can't keep up

Agent workflows are dynamic, but most access policies are fixed at deployment. This leads to excessive privileges, policy blind spots, and a lack of real-time enforcement.

No delegation or provenance tracking

Without runtime delegation (e.g., OAuth OBO) or execution graphs, it's impossible to trace agent actions back to users—creating compliance gaps and audit failures.

Identity sprawl at machine speed

Agents scale fast, often generating thousands of short-lived identities. Without automated lifecycle controls, credentials persist beyond their use, creating risk at scale.

IAM is siloed by system

Agents interact across APIs, MCPs, SaaS, and on-prem, but identity tools don't span domains. Without orchestration, policies remain fragmented and unmanageable.

Agentic AI demands a new identity architecture that supports ephemeral agents, real-time delegation, and policy enforcement across systems.

Solution: A runtime identity model for securing AI at scale

Identity isn't just for humans anymore — it's becoming the security backbone for intelligent, non-human agents operating at scale.

What's driving the shift?

Traditional IAM was built for people: long-lived accounts, logins, passwords, and access roles. However, agentic AI requires a new model built for **dynamic, autonomous, ephemeral actors** operating across distributed, hybrid environments. Agentic AI is:

- **Autonomous:** Acts without human prompts
- **Delegated:** Operates on behalf of users or systems
- **Distributed:** Runs across multi-cloud and on-prem systems

This shift demands a rethink of core identity functions: authentication, access control, authorization, audit, and governance.

Identity management in the agent era

Here's how identity must evolve to support secure, scalable AI agent architectures:

Agent authentication: Verifying digital actors in real time

Unlike users, agents don't log in; they authenticate using short-lived credentials tied to specific tasks.

- **SPIFFE/SVID** – signed X.509 certs
- **PKCE** – secure OAuth flows without secret sharing
- **mTLS + JWT** – verifiable session binding

Access control: Enforcing runtime guardrails

RBAC and static ABAC fall short for agents that change context constantly. Modern enforcement includes:

- Scoped, time-bound tokens
- Dynamic ABAC policies (task + user intent + risk)
- Policy-as-code via OPA or Cedar
- Ideally, it is enforced through an API gateway like Strata's App Fabric or MCP-aware proxies.

Authorization: Delegated identity via OAuth OBO

Agents acting on behalf of users require:

- OAuth On-Behalf-Of (OBO) flows
- Delegation tracking (user / agent /service)
- Signed claims for role, intent, and scope
- This enables trusted, traceable execution chains.

Auditing: Observability for agent behavior

Logging isn’t enough. Agent auditing requires:

- Execution graphs for multi-agent workflows
- Signed attestations
- Context-rich telemetry (who, what, on whose behalf)
- Feeds into SIEMs for real-time compliance validation.

Administration & governance: Just-in-time, policy-driven identity

Agent identities should be:

- Ephemeral and JIT-issued
- Scoped with TTL
- Managed via CI/CD pipelines

Registries must track metadata, scopes, and lifecycle events to prevent identity sprawl.

Key differences between human and agentic identity

Function	Human Identity	Agentic identity
Authentication	Login, MFA, SSO, biometrics	SPIFFE/SVID, PKCE, JWT, mTLS
Access Control	RBAC/ABAC, group membership	Scoped, task-aware, runtime permissions
Authorization	Session-based scopes	OBO delegation, signed role assertions
Auditing	SIEM event logs	Execution graphs, decision traceability
Governance	Manual provisioning, role reviews	JIT identity, policy-bound registry

The future of identity is runtime-driven and agent-aware

Identity is no longer about who logged in—but who (or what) is acting in real time. Agentic identity infrastructure, powered by **Identity Fabrics, Agent Registries, and Orchestration Layers**, enables organizations to:

- Trust agents at runtime
- Secure access dynamically
- Audit actions end-to-end

Platforms like **Strata’s Mavericks** make this a reality, extending Zero Trust into the age of agentic AI.

Chapter 5:

Governing identity at machine scale

Traditional IAM systems are built around the concept of a persistent user session. A person logs in, receives a token, and their actions are tracked throughout that session. This model works well for human users, but breaks down completely when applied to autonomous agents.

Problem: Why AI agents break traditional identity systems

The systems we use to manage identity today were designed for humans—long-lived, predictable users operating in stable environments. AI agents are the opposite: **ephemeral, autonomous, delegated, and fast**. That mismatch creates a growing identity crisis in the enterprise.

Static provisioning can't keep up

Agents often exist for seconds and spin up dynamically in response to tasks. Static identities and manual provisioning processes can't scale to meet this velocity.

Long-lived credentials increase risk

Without dynamic provisioning, teams fall back on shared secrets and pre-provisioned credentials. These are often over-permissioned, difficult to rotate, and impossible to audit effectively.

Delegation and traceability are missing

Agents often act on behalf of users or systems, but without a way to bind identity and intent at runtime, there's no clear chain of trust. That means no clean audit trail—and no way to answer "who authorized this?"

No standardized identity for agents

Some agents use API keys. Others impersonate service accounts. Few carry meaningful identity attributes like TTL, provenance, or risk tier. This inconsistency breaks Zero Trust enforcement.

Credential sprawl creates operational drag

Managing and deprovisioning static identities across thousands (or millions) of agents is expensive, fragile, and error-prone.

IAM can't scale to agent volumes

Traditional IAM architectures can't handle the scale, speed, or complexity at which agents are proliferating across the enterprise.

Agent behavior is hard to monitor or govern

Without policy-aware, runtime identity, there's no way to enforce least privilege, apply context-aware controls, or trace agent actions with confidence.

We're using a human-centric identity model for non-human actors. And it's not working. Without identity systems that match the dynamic, delegated, and ephemeral nature of agents, enterprises face operational overhead, rising risk, and compliance exposure they can't afford.

Solution: Just-in-time provisioning for agentic identity at runtime

AI agents are moving fast—and enterprise identity must move with them. Traditional IAM models assume identities are static and long-lived. But agents are **temporary, delegated, and task-specific**. They need credentials only for the moment they act—no sooner, no longer.

That's why the future of agent identity is **just-in-time (JIT) provisioning**.

What is JIT identity provisioning?

Just-in-Time provisioning creates an agent identity **only at runtime**, scoped precisely to the task at hand. When the task is complete, the identity is retired.

The result is no stale identities, least privilege by default, full traceability, and no credential sprawl.

How JIT works with Mavericks

Strata's Mavericks platform enables just-in-time identity provisioning by orchestrating agent identity at runtime—driven by policy, context, and delegation.

When a human or system initiates a task, Mavericks dynamically provisions a scoped identity for the AI agent assigned to carry it out. This identity is issued just in time, with only the permissions needed for the specific task. The agent authenticates using short-lived, purpose-bound credentials and executes the operation—whether that's making an API call, accessing data, or completing a transaction.

Once the task is complete, the agent identity is automatically retired or revoked. Throughout the process, Mavericks logs every action, including who delegated the task, what the agent did, and under what policy—all recorded for auditing and compliance.

This runtime flow ensures that agent identities are secure, ephemeral, and fully traceable—matching the speed and scale of AI without compromising on governance.

Key benefits of JIT provisioning for AI agent identity

- **Least privilege, always**
Agents get access only to what they need, for as long as they need it.
- **No credential sprawl**
JIT eliminates long-lived secrets and static agent accounts.
- **Dynamic scalability**
Scales to millions of agents without manual provisioning.
- **Compliance-ready by design**
All agent activity is logged, delegated, and auditable—supporting frameworks like DORA, GDPR, and NIST 2.0.

The future is runtime identity

As AI agents multiply, identity must adapt. That means **moving from static to dynamic**, and from pre-provisioned to **on-demand**. JIT provisioning is the only scalable way to secure AI agents without sacrificing agility, visibility, or compliance.

Strata's Mavericks platform makes this real—enabling enterprises to embrace agentic identity at scale with **no code rewrites, no lock-in, and no compromise**.

Chapter 6:

Extending OAuth for the agentic future

OAuth, the standard backbone of delegated access, is buckling under agentic pressure. It assumes human users, stable session contexts, and predictable lifecycles. Agentic AI is none of those. Agents cross domains, spawn at runtime, and act on behalf of humans, systems, and other agents — without manual logins or long-lived state.

Problem: Why OAuth needs to evolve for the agentic era

OAuth 2.0 changed the game for access delegation, but it wasn't built for ephemeral actors, multi-hop workflows, or distributed decision-making at runtime. Agentic AI creates demands that OAuth alone can't meet.

Delegation chains are ambiguous

OAuth supports simple delegation (user / app), but agents often act on behalf of users, other agents, or systems. These chains of trust are not explicitly represented in most OAuth implementations, leaving gaps in attribution and audit.

Tokens outlive agents

Agents may exist for seconds, yet OAuth tokens are typically valid for minutes or hours. This mismatch introduces risk: tokens remain usable after agents complete their task or disappear, enabling replay or misuse.

Cross-cloud trust propagation is missing

AI agents cross domains and clouds as part of normal workflows. But OAuth doesn't natively support token propagation across trust boundaries, forcing over-permissioned workarounds or brittle handoffs.

Public agents can't store secrets

Many agents are public clients—stateless LLM functions, browser-based copilots, or plugin agents. They can't store static client secrets, making them vulnerable to token interception without flows like PKCE.

No runtime enforcement or revocation

Once issued, OAuth tokens are valid until expiry. If an agent's behavior changes mid-execution—or a delegator revokes trust—there's no way to adapt. Traditional token lifetimes create blind spots in zero trust enforcement.

Scopes are too coarse for real-world delegation

OAuth scopes tend to be broad ("read profile," "write data") and hardcoded. Agents need dynamic, context-sensitive permissions based on risk, role, task, and time—not static entitlements.

No built-in observability

OAuth logs are minimal, and don't capture the full chain of who acted, on whose behalf, for what reason. Without rich telemetry, organizations can't answer essential questions like "Who authorized this?" or "What triggered that action?"

The result: OAuth without orchestration leads to identity chaos

Enterprises that rely on OAuth alone for agentic AI see over-permissioned tokens, untraceable delegation, and insufficient controls. The protocol isn't broken—but it needs orchestration to work at runtime.

Without a modern runtime identity layer, enterprises are exposed to credential misuse, blind delegation, and policy enforcement failures. And with AI agent populations projected to outpace human users 80:1, the risks only grow.

Solution: Extending OAuth for the agent era

OAuth is the right foundation for agentic identity—but it's not the full answer. AI agents operate at machine speed, across systems, and often without human supervision. To secure them, identity must move from static to dynamic, and from issued-once to evaluated continuously.

Strata's Mavericks platform makes this shift real, by orchestrating OAuth flows at runtime, enforcing delegation policies in context, and logging every step for audit and compliance.

On-Behalf-Of (OBO): Bind agents to their delegators

Maverics implements OAuth OBO flows to bind every agent action to its originator—human, agent, or system. This creates a verifiable chain of intent and makes delegated authorization auditable in real time.

Token exchange: Secure trust across clouds

Maverics uses OAuth token exchange (RFC 8693) to propagate identity across domains and platforms. This eliminates the need to over-permission agents or hardcode trust boundaries—tokens stay scoped and transportable.

DPoP: Proof-of-possession stops token theft

For environments with high churn and limited trust, Mavericks uses DPoP (Demonstration of Proof-of-Possession) to cryptographically bind tokens to the specific agent presenting them. If the token is intercepted, it's unusable without the agent's key.

PKCE: Secure flows for public agents

Maverics protects agents that can't store secrets (e.g., browser agents or LLMs) with PKCE. This prevents authorization code interception and ensures public clients authenticate securely without shared secrets.

CAEP: Enforce access dynamically

Static tokens are dangerous in dynamic environments. Mavericks integrates with CAEP (Continuous Access Evaluation Protocol) signals to revoke or re-authorize tokens at runtime—based on behavior, risk, or policy triggers.

Attribute-based authorization: Context-aware policy

Beyond scopes, Mavericks evaluates token claims, agent metadata, and environmental context to make fine-grained access decisions. This supports task-specific delegation, agent TTL, and human-in-the-loop approvals.

Full observability: Execution graphs and audit

Every identity operation is logged: who acted, what was authorized, what scopes were used, and why. These logs feed your SIEM via OpenTelemetry, enabling traceability across multi-agent workflows and automated forensics.

Key benefits of Mavericks for OAuth-based agent identity

Maverics extends and operationalizes OAuth for AI agents with a set of critical enhancements, each built to handle the complexity of delegated, distributed, ephemeral actors.

- **Secure delegation:** Every agent action is tied to its delegator—verifiably, traceably, and securely.
- **Zero trust by design:** Access is enforced based on real-time context, not static roles or scopes.
- **No static secrets:** PKCE and DPoP eliminate the need for long-lived credentials.
- **Cross-cloud identity propagation:** Token exchange supports agents operating across clouds and APIs.
- **Audit built in:** Execution graphs and telemetry provide a full picture of agent behavior and intent.

The future is orchestrated OAuth

OAuth remains a powerful protocol, but agentic AI demands more. With Mavericks, OAuth becomes a dynamic identity foundation: delegated, ephemeral, scoped, auditable, and enforced in real time.

As enterprises scale agent adoption, only orchestration can extend OAuth to meet the complexity of modern workflows. Without sacrificing ZeroTrust, compliance, or speed.

Chapter 7:

Agents are people too

Most systems still treat agents like background processes or glorified apps. That gap is now one of the biggest risks facing enterprise adoption of agentic AI.

Problem: Identity systems still assume people

Without a purpose-built identity model, agents often operate invisibly. They're spun up without being registered in an IDP. They inherit broad privileges instead of having scoped, task-specific ones. They complete actions, but there's no way to trace what they did or who authorized them.

Worse, many agents use hardcoded credentials or shared service accounts, creating a blind spot in enforcement and a goldmine for attackers.

Agents aren't treated as first-class identities

Enterprises are deploying increasingly capable AI agents, but their IAM infrastructure still treats these agents as background processes or shared service accounts. Most agents today don't have unique, managed identities. Without distinct digital identities. That gap creates real risks.

When agents aren't treated as first-class identities:

- Least privilege breaks down.
- Delegation becomes untraceable.
- Policy enforcement is coarse at best, and nonexistent at worst.
- Audit trails are patchy, leaving compliance and incident response on shaky ground.

As agents become more autonomous, these gaps will only widen. Without identity systems built for the way agents actually operate, innovation will slow, risk will grow, and governance will falter.

It's time to stop treating AI agents like edge cases. They're core to how modern enterprises work and need identity guardrails to match.

Solution: Identity Orchestration built for agents

Maverics gives each agent a just-in-time identity—created at runtime, scoped to its task, and automatically retired when it's done. These ephemeral identities are tightly bound to the delegator, whether that's a human, another agent, or a system.

Instead of static roles or broad scopes, Maverics evaluates policy in real time. Authorization decisions factor in task purpose, context, risk signals, and delegation history—enforcing Zero Trust without over-permissioning or manual policy sprawl.

For sensitive actions, Maverics can require human approval mid-workflow. Liveness checks, passwordless MFA, and step-up prompts help enterprises maintain control when it matters most—without disrupting automation where it doesn't.

Identity that works at machine speed

Behind the scenes, Mavericks orchestrates the full range of modern OAuth flows:

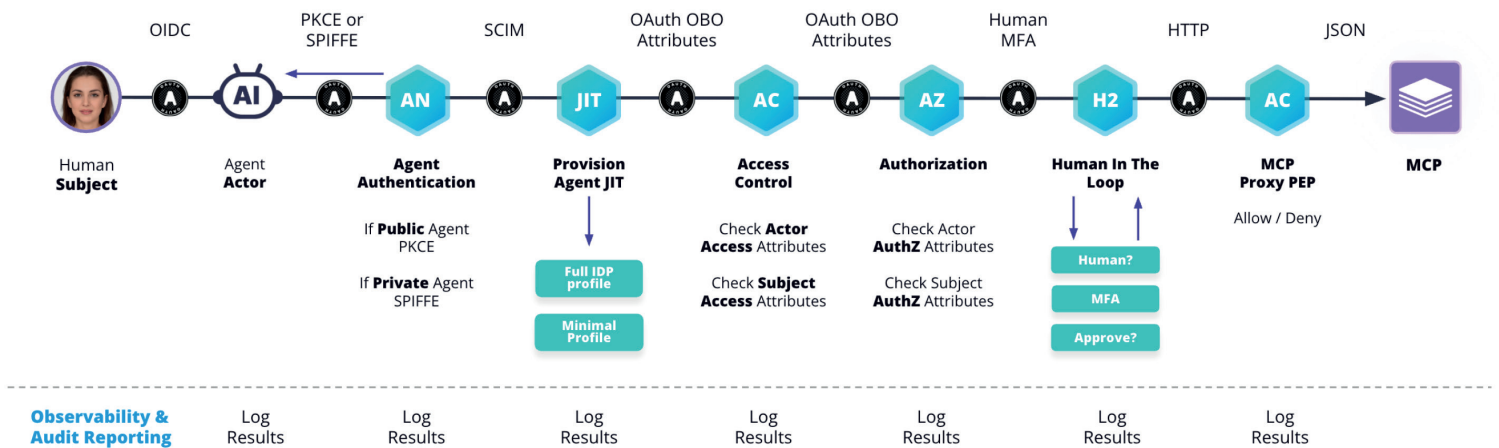
- **OBO** binds agent actions to their source.
- **PKCE and DPoP** secure agents without shared secrets.
- **Token exchange and cross-cloud federation** enable agents to move safely across domains.
- **CAEP** allows for continuous enforcement and revocation based on live risk conditions.

And everything is observable. Mavericks generates rich telemetry for every agent action — who acted, what was done, under what delegation, and why it was allowed. This data integrates directly with existing SIEMs via OpenTelemetry, supporting forensic reconstruction and compliance with ease.

Why it matters now

The number of agents in the enterprise is exploding. Some teams already manage 80x more agents than humans. Without automated, auditable identity for agents, the risk becomes unmanageable.

Maverics helps enterprises scale agent adoption with confidence—governing AI actors as first-class identities, enforcing policy in real time, and logging everything for audit and analysis.



Chapter 8:

The identity gaps in agentic AI

Many of today's agents operate in a dangerous middle ground: they are trusted to act but not treated like true identities. That gap creates risks that we'll explore below in the final chapter.

Problem:

Most IAM architectures were built around humans and static machines. AI agents are neither. They're ephemeral, task-driven, and delegated — behaviors legacy IAM can't accommodate.

For example, most systems can't securely bind a human or system to an agent it has delegated. Agents are often spun up without being registered, authenticated using shared secrets, or granted overly broad access because fine-grained control doesn't exist.

When something goes wrong, it's hard to know who initiated the action, what the agent was authorized to do, or whether policy enforcement occurred at all.

And that's just the beginning.

Agent flows frequently cross domains and APIs, but identity can't follow them. Delegation is vague. Authorization is coarse. Sensitive actions lack human validation. And even if the system works, logs don't capture the full context—leaving security teams without a clear record of what happened.

The breakdowns in the agentic identity lifecycle

While the symptoms vary, the root cause is the same: our identity infrastructure wasn't built for dynamic, autonomous actors. Key problems include:

- No trustworthy authentication between delegators and agents
- Blurry delegation chains and unclear scopes
- Inability to express or enforce intent
- Static credentials or missing agent registration entirely
- Lack of context-aware discovery and real-time policy evaluation
- No human-in-the-loop for sensitive operations
- Insufficient logging for forensics or compliance

Individually, these issues cause friction and security gaps. Together, they stall agent adoption—especially in regulated or high-impact environments.

Why this can't wait

Agent-based automation is growing fast. Gartner predicts nearly one-third of enterprises will rely on AI agents for key workflows within the next year. But without an identity foundation purpose-built for them, enterprises face:

- Over-permissioned agents and exposed APIs
- Compliance failures due to unverifiable delegation
- Innovation bottlenecks as teams restrict agent use out of caution

To scale securely, organizations need to treat agent identity as a first-class discipline, not a bolt-on to human IAM.

Solution: A new identity playbook for AI agents

As AI agents take on more responsibility across enterprise systems, one thing is clear: they need identity models as dynamic as they are. Traditional non-human identity (NHI) approaches—like service accounts and static API keys — can't scale to govern actors that think, delegate, and act autonomously.

That's why Strata built Mavericks Agentic Identity. It's not just an adaptation of IAM, it's a new architecture designed for a new class of identity.

Real-time identity orchestration for autonomous actors

With Mavericks, agents are no longer invisible background processes. They're provisioned as dynamic, policy-bound identities at runtime, fully observable, controllable, and integrated into the Zero Trust fabric.

Here's how a modern agentic user flow works:

- 1. Establish trust.** A human or system delegates a task to an agent. The agent authenticates using secure, secret-less methods like PKCE or SPIFFE, depending on its trust level.
- 2. Provision just-in-time.** Mavericks creates an identity for the agent only when needed. This identity is tightly scoped to its purpose, with attributes like TTL, risk tier, and delegation chain attached.
- 3. Evaluate access continuously.** Mavericks enforces layered policies at runtime, combining coarse-grained API-level rules with fine-grained ABAC or OPA-based logic. Everything is evaluated in context—purpose, risk, time, and more.
- 4. Involve humans where needed.** For sensitive workflows, Mavericks inserts human validation steps: liveness checks, passwordless MFA, or explicit approval before agents can proceed.
- 5. Capture everything.** Every step—from delegation to execution—is logged and correlated. Logs integrate via OpenTelemetry, giving teams a full execution graph for audit and compliance.

This model replaces brittle, pre-provisioned NHI patterns with something far more secure and scalable: real-time trust decisions based on identity, policy, and context.

Why agentic identity is different

AI agents may technically fall under the category of non-human identities, but functionally, they operate in an entirely different class. Traditional NHIs—like service accounts, API clients, or machine users—are static, narrowly scoped, and often tied to a single system or task. In contrast, agentic identities are dynamic, autonomous, and fluid.

They reason, delegate, and act independently—often across domains and systems—requiring real-time policy evaluation, accountability, and human oversight for sensitive actions. Securing these identities demands a fundamentally new approach, not just an extension of legacy NHI models.

Unlike traditional machine identities, agents:

- Don't persist—they spin up and down on demand
- Act on behalf of others—creating complex trust chains
- Require governance across domains and clouds
- Often execute workflows independently of human users

That means they need JIT provisioning, delegated authorization, dynamic policy enforcement, and runtime observability — not static roles or long-lived credentials.

<i>Agentic Identity</i>	<i>Non-Human Identity (NHI)</i>
Ephemeral, JIT created	Long-lived, pre-provisioned
Acts autonomously, often with delegation chains	Static purpose, tied to a single app/system
Requires dynamic policy evaluation	Managed via fixed roles/scopes
Works across domains and trust zones	Typically scoped to a single system
Needs human-in-the-loop for sensitive tasks	Rarely includes live human validation
Bound by Zero Trust and least privilege	Often over-permissioned or coarse-grained

The Mavericks advantage for managing AI agents at scale

With Mavericks, enterprises gain:

- Scoped, ephemeral identities for every agent
- Delegation chains that are explicit and auditable
- Context-aware access control that adapts to risk
- Human-in-the-loop protection for high-risk actions
- End-to-end visibility across every agent interaction

In short, Mavericks brings order to agentic complexity, without slowing agents down or forcing changes to existing apps and IDPs.

Conclusion

The future of identity is agentic

AI agents are no longer on the horizon—they're already reshaping how enterprises operate. From autonomous copilots to multi-agent workflows that span APIs, clouds, and systems, these actors are driving decisions and executing tasks at machine speed. But their rise has outpaced the identity infrastructure meant to govern them.

What we're seeing now is a turning point. The old model—designed for humans and static non-human identities—cannot handle the scale, speed, or complexity of agentic AI. Identity gaps are becoming attack surfaces. Delegation is murky. Audit trails are broken. And without policy-aware identity systems built for runtime orchestration, the risks will only grow.

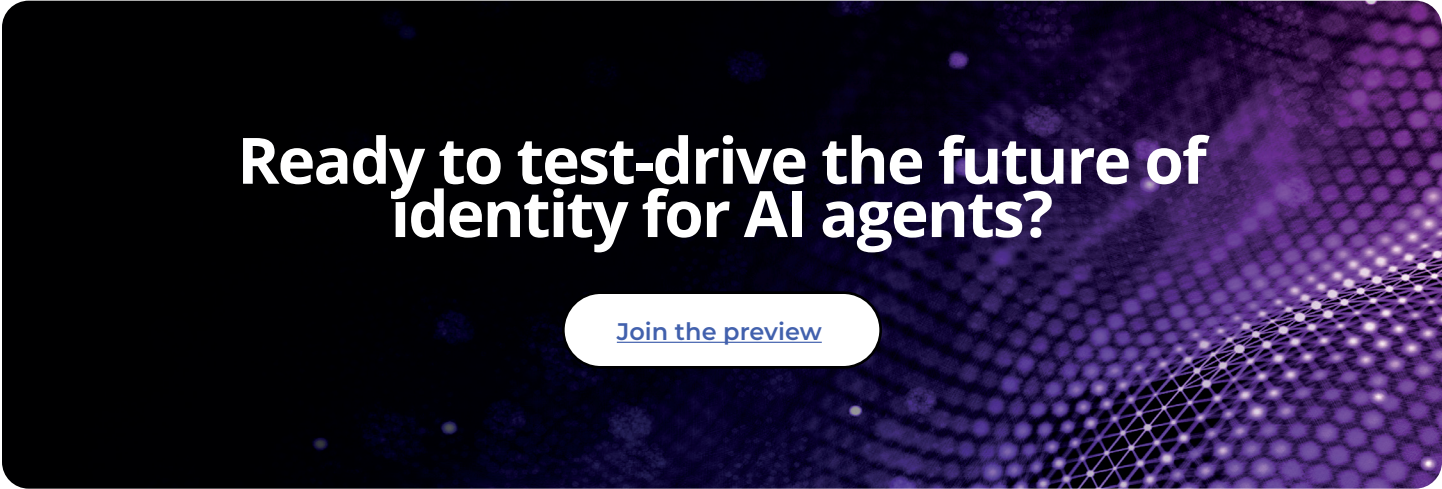
Strata's Mavericks platform addresses this head-on. By treating agents as first-class identities, enabling just-in-time provisioning, orchestrating OAuth at scale, and enforcing Zero Trust policies in real time, Mavericks offers the foundation enterprises need to govern agents without slowing them down.

This isn't just a new feature set — it's a new identity playbook for a new class of actors.

Because the future isn't static identity. It's dynamic, contextual, and agent-driven. And it's already here.

Now is the time to build the identity architecture that can keep up.

Join the early access program for Identity Orchestration for AI Agents and help shape the next generation of secure, scalable AI identity.



**Ready to test-drive the future of
identity for AI agents?**

[Join the preview](#)